

M1 - MASTER SAR

Janvier 2016

2 heures - Tout document papier autorisé

Barème sur 21 points donné à titre indicatif

L. Arantes, P. Cadinot, E. Encrenaz, S. Monnet, P. Sens, J. Sopena

I. QUESTIONS DE COURS (4 POINTS)

On considère deux processus P1 et P2. P1 est plus prioritaire que P2. Soit le scénario suivant :

- A l'instant t, P1 est prêt et P2 bloqué sur la priorité PRIBIO. P1 est le processus courant.
- A t+2ms, P1 fait un appel système qui doit durer 5ms de temps CPU.
- A t+3ms, une interruption disque survient, son traitement dure 1ms et réveille le processus P2.
- A son réveil le processus P2 fait un traitement qui dure 1ms puis se rendort. On néglige les temps de commutations.

I.1. (1,5 points)

En considérant un noyau **non préemptif**, faites un schéma temporel (diagramme de Gantt) représentant les processus courants à partir de t. A quel moment, P1 termine son appel système ?

I.2. (1,5 points)

Reprendre la question précédente en considérant un noyau **préemptif**.

I.3. (Question indépendante - 1 point)

- (a) Un tick peut-il être plus petit qu'un quantum ?
- (b) Une commutation peut-elle avoir lieu avant la fin d'un quantum, si oui dans quels cas ?

II. SYNCHRONISATION (6 POINTS)

Nous voulons offrir la fonction système `int sum_sync(int val)` qui réalise une somme synchronisée parmi les processus fils d'un même processus. La fonction rend la somme des valeurs passées dans le paramètre `val` par chacun des processus fils.

Exemple : supposons 3 processus fils.

fils ₁	fils ₂	fils ₃
... sum_sync (2); sum_sync (3);	... sum_sync (4); sum_sync (2);	... sum_sync (1); sum_sync (4);

Pour les 3 processus fils, le premier et le deuxième appels à `sum_sync` renvoient 7 et 9 respectivement.

Nous considérons que les programmes utilisent correctement `sum_sync(int val)`, autrement dit, tous les processus fils appellent le même nombre de fois la fonction.

Si un processus s'endort, sa priorité de réveil sera P_WAIT.

II.1. (3 points) → dormir sur l'addr du suivant

Donnez le code de l'appel système `_sum_sync` appelé par la fonction `sum_sync(int val)` en spécifiant les champs que vous allez ajouter dans les structures du noyau et sur quelle adresse les processus vont s'endormir.

Nous considérons maintenant que chacun de N processus fils a un identifiant unique `id` (`id=1, 2, ... N`) qu'il connaît. Nous voulons implémenter la fonction `multi_sum_sync (int id, int val)` qui rend la valeur :

$$((((((1*id_1)+val_1)*id_2+val_2)*id_3+val_3)*...)*id_N)+val_N.$$

Autrement dit, chaque fils *i* doit multiplier le résultat produit par le fils `idi-1`, *i*=2...N, (ou 1, dans le cas du premier fils) par son `id` (`idi`) et additionner la valeur de `val` passé en paramètre. Notez que les opérations effectuées par chaque processus doivent être faites dans l'ordre croissant de leurs `ids`.

Exemple : supposons 3 processus fils.

fils ₁	fils ₂	fils ₃
... multi_sum_sync (1,2); multi_sum_sync (1,3);	... multi_sum_sync (2,4); multi_sum_sync (2,2);	... multi_sum_sync (3,1); multi_sum_sync (3,4);

Le premier appel rend $(((1*1)+2)*2)+4)*3)+1= 31$ et le deuxième: $(((1*1)+3)*2)+2)*3)+4=34$

II.2. (3 points)

Donnez le code de l'appel système `_multi_sum_sync` appelé par la fonction `multi_sum_sync(int id, int val)` en spécifiant les champs que vous allez ajouter dans les structures du noyau et sur quelle adresse les processus vont s'endormir.

III.FICHIERS (7 POINTS)

Nous considérons une partition organisée de la manière suivante : un *super-bloc* étendu sur 3 blocs, des blocs contenant des *inodes* (32 *inodes* par bloc) puis les blocs de données. Les blocs sont numérotés de 0 à n et les *inodes* de 1 à m. Les blocs font 1024 octets et les numéros de blocs sont codés sur 32 bits (un bloc peut donc contenir 256 numéros de blocs). Enfin, nous considérons que les *inodes* ont une structure équivalente à celle vue en TD (13 entrées dans l'*inode*, les 10 premières pointant sur des blocs de données, la onzième sur un bloc d'adresses, la douzième offrant une double indirection et la treizième, une triple indirection).

III.1.(0,5 point)

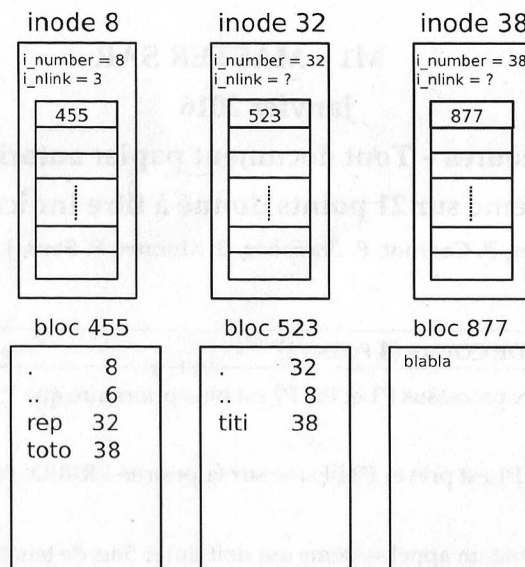
Quelle est la taille maximale d'un fichier ?

A présent considérons le code suivant :

```
00 : int f1,f2,f3 ;
01 : char[4] buffer ;
02 : f1 = open("../toto.txt", O_RDWR) ;
03 : f2 = open("../rep/titi.txt", O_RDONLY) ;
04 : if(fork()==0) {
05 :   int f3 = open("../toto.txt", O_RDWR) ;
06 :   write(f1,"bonjour",8) ;
07 :   write(f3,"bonne année",12) ;
08 :   close(f1) ; close(f2) ; close(f3) ;
09 :   exit(0) ;
10 : } else {
11 :   wait(0) ;
12 :   read(f1,buffer,4) ;
13 :   printf("%s",buffer) ;
14 :   close(f1) ; close(f2) ;
15 :   exit(0) ;
16 : }
```

Nous faisons l'hypothèse que seul l'*inode* du répertoire courant (*inode* 8) est dans la table `inode[]`. Aucun des blocs concernés n'est en mémoire.

Nous considérons les *inodes* et les blocs suivants :



III.2.(1 point)

Quel est l'affichage produit (justifier) ?

III.3.(2,5 points)

Donner l'état, juste après la ligne 7, des tables de descripteurs de fichiers, fichiers ouverts, et *inodes* (faire un schéma, penser à représenter notamment les champs `f_count`, `f_offset`, `i_count`, `i_number` et `i_nlink`).

III.4.(1 point)

Dans quels blocs disque se situent les *inodes* 8, 32 et 38 (préciser la formule utilisée) ?

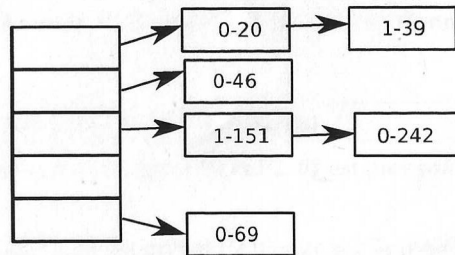
III.5.(2 points)

- Donner la liste des blocs accédés jusqu'à la fin de la ligne 7 (expliquer).
- En considérant que la `bfreelist` du buffer cache contient 10 blocs libres (aucun signalé avec écriture différée), combien d'opérations E/S sont réalisées lors de ces accès (nous considérons que les opérations d'écriture ne sont pas différées).

IV. BUFFER-CACHE (4 POINTS)

Nous considérons un buffer cache de 6 cases, ayant un tableau *hbuf* de 5 cases et utilisant comme fonction de hash : $(device_number + bloc_number) \% 5$.

L'état initial est décrit par la figure suivante.



Représentation : devno-blkno

b_freelist : 0-69, 0-46, 1-39, 0-242, 0-20, 1-151

On considère également la séquence d'accès suivante :

1-39, 0-42, 0-69, 1-151, 0-463

IV.1.(2 points)

- Donner l'évolution de la *b_freelist* à chaque accès.
- Donnez la nouvelle configuration du buffer cache après la séquence d'accès (en considérant que seuls ces accès ont lieu).
- Combien d'entrées/sorties sur disque sont générées par cette séquence ?

IV.2.(2 points)

On considère maintenant que le buffer 0-46 est en écriture différée (*DELWRI*). En partant du même état initial, avec la même séquence, que pour la question précédente, donner l'état final de la *b_freelist*, du buffer cache et le nb d'E/S.